

Presentación 8, 9 y 10

Nombres de los estudiantes:

Jhon Alexander Terreros Vargas – ID : 892565

Juan Felipe Meneses Rodriguez – ID : 1022304

Institución: UNIMINUTO

Programa: Ingeniería de Sistemas

Curso: Diseño de Algoritmos

Docente: Esteban Ernesto Morales Castro

Fecha: 027/04/2026

1. Tabla De Contenidos

Contenido

1. Tabla De Contenidos	2
3. RESUMEN	5
4. PALABRAS CLAVE.....	5
5. INTRODUCCIÓN.....	5
6. OBJETIVOS	6
6.1 Objetivo General	6
6.2 Objetivos Específicos	6
7. METODOLOGÍA	6
8. ESTRUCTURA DEL DOCUMENTO	6
9. Problema 1 - GREEDY (Cajero Automático).....	7
Un cajero debe dispensar \$87,500 COP usando billetes de [50000, 20000, 10000, 5000, 1000]..	
• Implementa el algoritmo greedy en Python o Java	7
• Muestra la traza completa paso a paso	7
• ¿Qué pasa si agregas un billete de \$7000? ¿Sigue siendo óptimo?	7
• Calcula la complejidad de tu solución.....	7
9.1 Algoritmo (Explicación)	7
9.2 Tabla.....	7
9.3 Resultado.....	7
9.4 Código.....	8
9.5 Análisis: ¿Qué pasa si agregamos 7000?	9
9.6 Complejidad	9
9.7 Análisis Problema 1	9
10. Problema 2 – HUFFMAN (Compresión de logs)	10
Un log repite las siguientes palabras: ERROR(45), INFO(120), WARN(30), DEBUG(80), TRACE(15).	
• Construye el árbol de Huffman paso a paso	10
• Asigna códigos binarios a cada palabra.....	10
• Calcula bits usados sin y con compresión	10
• ¿Qué porcentaje de espacio se ahorra?	10

10.1	Algoritmo (Explicación)	10
10.2	Construcción del árbol (paso a paso)	10
	• Paso 1: TRACE(15) + WARN(30) = 45	10
	• Paso 2: 45 + ERROR(45) = 90	10
	• Paso 3: DEBUG(80) + 90 = 170	10
	• Paso 4: INFO(120) + 170 = 290 (raíz)	10
10.3	Códigos binarios resultantes	11
10.4	Cálculo de bits	11
	Sin compresión	11
	Supongamos 8 bits por palabra:	11
	Con Huffman	11
11.	Código	12
11.1	Complejidad	13
11.2	Análisis Problema 2	13
12.	PROBLEMA 3 – KNAPSACK 0/1 (Programación Dinámica)	14
12.1	¿Cómo funciona?	14
12.2	Fórmula clave	14
12.3	Tabla	14
12.4	Explicación de algunas celdas clave	15
12.5	Resultado óptimo	15
12.6	Backtracking (cómo se obtiene la solución)	15
12.7	Proyectos seleccionados	15
12.8	Código	16
12.9	Complejidad	17
	• Tiempo: $O(n \times W)$	17
	• Espacio: $O(n \times W)$	17
12.10	Análisis Problema 3	17
13.	Problema 4 – LCS (Subsecuencia Común Más Larga)	17
13.1	Idea del Algoritmo	17
13.2	Regla del algoritmo	17
13.3	Cadenas	18

13.4	Resultado de la LCS	18
13.5	Tabla DP	18
13.6	Backtracking (Reconstrucción)	19
13.7	Código	20
13.8	¿Qué significa esta LCS en el contexto del versionado?	21
13.9	Complejidad	21
13.10	Análisis del Problema 4	21
14.	Referencias	22

3. RESUMEN

Este trabajo presenta la aplicación de algoritmos de tipo greedy y programación dinámica en la resolución de problemas reales de ingeniería de sistemas. Se abordan cuatro casos específicos: el problema del cajero automático, la compresión de datos mediante el algoritmo de Huffman, la optimización de inversión con el problema de la mochila 0/1, y la comparación de cadenas mediante la subsecuencia común más larga (LCS).

Para cada problema se desarrolla su respectiva solución, incluyendo la formulación del algoritmo, su implementación en código, el análisis de complejidad y la interpretación de resultados.

Además, se evalúa la eficiencia de cada enfoque y su aplicabilidad en contextos reales.

Los resultados obtenidos evidencian la importancia de seleccionar el paradigma algorítmico adecuado según la naturaleza del problema, destacando las ventajas y limitaciones de cada técnica.

4. PALABRAS CLAVE

Algoritmos, Greedy, Programación Dinámica, Optimización, Complejidad, Huffman, Knapsack, LCS.

5. INTRODUCCIÓN

En el ámbito de la ingeniería de sistemas, la resolución eficiente de problemas es fundamental para el desarrollo de sistemas robustos y escalables. Los algoritmos constituyen la base de esta resolución, permitiendo transformar datos de entrada en resultados útiles mediante procesos bien definidos.

Existen diversos paradigmas de diseño de algoritmos, entre los cuales destacan los algoritmos greedy (avidos) y la programación dinámica. Los primeros se caracterizan por tomar decisiones locales óptimas con la esperanza de alcanzar una solución global óptima, mientras que la programación dinámica se enfoca en descomponer problemas complejos en subproblemas más simples y reutilizar sus soluciones.

El presente trabajo tiene como objetivo aplicar estos paradigmas en situaciones reales, analizando su comportamiento, eficiencia y limitaciones. A través de cuatro problemas representativos, se busca comprender cómo estas técnicas contribuyen a la optimización y toma de decisiones en contextos prácticos.

6. OBJETIVOS

6.1 Objetivo General

Aplicar algoritmos greedy y programación dinámica a problemas reales de ingeniería, implementando y analizando sus soluciones.

6.2 Objetivos Específicos

- Implementar soluciones algorítmicas en un lenguaje de programación.
- Analizar la eficiencia de cada algoritmo mediante su complejidad.
- Comparar diferentes enfoques de resolución de problemas.
- Interpretar los resultados obtenidos en cada caso.
- Comprender las ventajas y limitaciones de cada paradigma.

7. METODOLOGÍA

Para el desarrollo de este trabajo se utilizó un enfoque práctico basado en la implementación de algoritmos en Python. Cada problema fue analizado inicialmente desde el punto de vista teórico, identificando el paradigma más adecuado para su solución.

Posteriormente, se diseñaron los algoritmos correspondientes, los cuales fueron implementados y probados. Se realizó un análisis de complejidad para cada solución, evaluando su eficiencia en términos de tiempo y espacio.

Finalmente, se interpretaron los resultados obtenidos, destacando el comportamiento de cada algoritmo y su aplicabilidad en escenarios reales.

8. ESTRUCTURA DEL DOCUMENTO

El documento se encuentra organizado de la siguiente manera:

- Problema 1: Aplicación de algoritmo greedy en un sistema de cajero automático
- Problema 2: Compresión de datos mediante el algoritmo de Huffman
- Problema 3: Optimización de inversión usando programación dinámica (Knapsack 0/1)
- Problema 4: Análisis de subsecuencia común más larga (LCS)

Cada sección incluye la explicación del problema, la solución propuesta, su implementación y el análisis de complejidad.

9. Problema 1 - GREEDY (Cajero Automático)

Un cajero debe dispensar \$87,500 COP usando billetes de [50000, 20000, 10000, 5000, 1000].

- Implementa el algoritmo greedy en Python o Java
- Muestra la traza completa paso a paso
- ¿Qué pasa si agregas un billete de \$7000? ¿Sigue siendo óptimo?
- Calcula la complejidad de tu solución

9.1 Algoritmo (Explicación)

El algoritmo recorre los billetes de mayor a menor denominación y:

1. Calcula cuántos billetes caben en el monto actual
2. Resta ese valor al monto
3. Continúa con el siguiente billete

9.2 Tabla

Billete	Cantidad usada	Monto restante
50000	1	37500
20000	1	17500
10000	1	7500
5000	1	2500
1000	2	500

9.3 Resultado

- 1×50000
- 1×20000
- 1×10000
- 1×5000
- 2×1000

Resto: 500 (no se puede representar)

9.4 Código

```
Problema_1.py > ...
1  def cajero_greedy(monto, billetes):
2      """
3      Simulación de un cajero automático que entrega el menor número
4      de billetes posible usando una estrategia voraz.
5      """
6      # Diccionario para almacenar cuántos billetes de cada denominación se entregan
7      resultado = {}
8
9      # El algoritmo asume que 'billetes' está ordenado de mayor a menor
10     for billete in billetes:
11         # Calculamos cuántas veces cabe el billete actual en el monto restante
12         # Usamos // para obtener la división entera
13         cantidad = monto // billete
14
15         if cantidad > 0:
16             # Guardamos en el diccionario: denominación -> cantidad entregada
17             resultado[billete] = cantidad
18             # Actualizamos el monto restante restando lo que ya vamos a entregar
19             monto -= cantidad * billete
20
21     # Retornamos el desglose y lo que no se pudo entregar (si sobra algo)
22     return resultado, monto
23
24
25 # --- Configuración de datos ---
26
27 # Es vital que la lista esté en orden descendente para que la lógica greedy funcione
28 billetes = [50000, 20000, 10000, 5000, 1000]
29 monto = 87500
30
31 # Llamada a la función
32 resultado, resto = cajero_greedy(monto, billetes)
33
34 # --- Salida de resultados ---
35 print("Distribución de billetes:", resultado)
36 # En este caso: {50000: 1, 20000: 1, 10000: 1, 5000: 1, 1000: 2}
37 print("Resto:", resto)
38 # El resto será 500 porque no hay billetes menores a 1000 🐛
```


9.5 Análisis: ¿Qué pasa si agregamos 7000?

Si se añade un billete de **7000**, el algoritmo greedy:

- Puede elegir 7000 en momentos donde no conviene
- Puede generar combinaciones con más billetes o no óptimas

Esto ocurre porque:

- Greedy solo toma decisiones locales, no evalúa todas las combinaciones posibles.

Conclusión:

- El sistema original funciona bien porque es canónico, pero agregar 7000 rompe esa propiedad.

9.6 Complejidad

- **Tiempo:** $O(n)$ → recorre lista de billetes
- **Espacio:** $O(n)$ → almacena resultados

9.7 Análisis Problema 1

El algoritmo greedy resulta eficiente para resolver el problema del cajero automático debido a la estructura de las denominaciones de los billetes, las cuales permiten obtener una solución óptima mediante decisiones locales. Sin embargo, al modificar el sistema agregando nuevas denominaciones como 7000, se pierde la propiedad de optimalidad, evidenciando una limitación importante de este tipo de algoritmos.

10. Problema 2 – HUFFMAN (Compresión de logs)

Un log repite las siguientes palabras: ERROR(45), INFO(120), WARN(30), DEBUG(80), TRACE(15).

Palabra	Frecuencia
ERROR	45
INFO	120
WARN	30
DEBUG	80
TRACE	15

- Construye el árbol de Huffman paso a paso
- Asigna códigos binarios a cada palabra
- Calcula bits usados sin y con compresión
- ¿Qué porcentaje de espacio se ahorra?

10.1 Algoritmo (Explicación)

Huffman es un algoritmo greedy que:

1. Toma los dos elementos de menor frecuencia
2. Los une en un nodo
3. Repite hasta formar un árbol

Los nodos más frecuentes quedan con códigos más cortos.

10.2 Construcción del árbol (paso a paso)

Orden inicial:

TRACE(15), WARN(30), ERROR(45), DEBUG(80), INFO(120)

- Paso 1: $\text{TRACE}(15) + \text{WARN}(30) = 45$
- Paso 2: $45 + \text{ERROR}(45) = 90$
- Paso 3: $\text{DEBUG}(80) + 90 = 170$
- Paso 4: $\text{INFO}(120) + 170 = 290$ (raíz)

10.3 Códigos binarios resultantes

Palabra	Código
INFO	0
DEBUG	10
ERROR	110
TRACE	1110
WARN	1111

10.4 Cálculo de bits

Sin compresión

Supongamos 8 bits por palabra:

Total palabras = $45 + 120 + 30 + 80 + 15 = 290$

Bits: $290 \times 8 = 2320$ bits

Con Huffman

Palabra	Frecuencia	Bits	Total
INFO	120	1	120
DEBUG	80	2	160
ERROR	45	3	135
TRACE	15	4	60
WARN	30	4	120

Total: $120 + 160 + 135 + 60 + 120 = 595$ bits

Ahorro = $(2320 - 595) / 2320 \times 100 \approx 74.35\%$

11. Código

```
Problema_2.py > ...
1 import heapq # Librería para manejar colas de prioridad (min-heaps)
2
3 # Definición de la estructura del árbol de Huffman
4 class Nodo:
5     def __init__(self, freq, simbolo=None, izq=None, der=None):
6         self.freq = freq # Frecuencia de aparición del símbolo
7         self.simbolo = simbolo # El carácter o palabra (ej: "INFO")
8         self.izq = izq # Hijo izquierdo
9         self.der = der # Hijo derecho
10
11     # Método para comparar nodos; necesario para que heapq sepa cuál es menor
12     def __lt__(self, otro):
13         return self.freq < otro.freq
14
15
16 def construir_huffman(frecuencias):
17     """
18     Crea el árbol binario de Huffman a partir de un diccionario de frecuencias.
19     """
20     # 1. Crear una lista de objetos Nodo y convertirla en una cola de prioridad (heap)
21     heap = [Nodo(freq, simbolo) for simbolo, freq in frecuencias.items()]
22     heapq.heapify(heap)
23
24     # 2. Mientras queden más de un nodo en el heap, seguimos uniendo los más pequeños
25     while len(heap) > 1:
26         # Extraemos los dos nodos con las frecuencias más bajas
27         n1 = heapq.heappop(heap)
28         n2 = heapq.heappop(heap)
29
30         # Creamos un nuevo nodo "padre" cuya frecuencia es la suma de sus hijos
31         # Los hijos son n1 (izquierda) y n2 (derecha)
32         nuevo = Nodo(n1.freq + n2.freq, None, n1, n2)
33
34         # Insertamos el nuevo nodo de vuelta en la cola
35         heapq.heappush(heap, nuevo)
36
37     # El último nodo que queda es la raíz del árbol completo
38     return heap[0]
39
40
41 def generar_codigos(nodo, prefijo="", codigos={}):
42     """
43     Recorre el árbol de forma recursiva para asignar ceros y unos.
44     """
45     # Si el nodo tiene un símbolo, es una "hoja", guardamos su código binario
46     if nodo.simbolo:
47         codigos[nodo.simbolo] = prefijo
48     else:
49         # Si no es hoja, vamos a la izquierda (añadimos '0')
50         # y a la derecha (añadimos '1')
51         generar_codigos(nodo.izq, prefijo + "0", codigos)
52         generar_codigos(nodo.der, prefijo + "1", codigos)
53     return codigos
54
55
56 # --- Datos de prueba (Niveles de logs) ---
57 frecuencias = {
58     "ERROR": 45,
59     "INFO": 120,
60     "WARN": 30,
61     "DEBUG": 80,
62     "TRACE": 15
63 }
64
65 # Ejecución:
66 arbol = construir_huffman(frecuencias)
67 codigos = generar_codigos(arbol)
68
69 print("Códigos Huffman:", codigos) 🐛
```

11.1 Complejidad

- **Tiempo:** $O(n \log n)$ → uso de heap
- **Espacio:** $O(n)$

11.2 Análisis Problema 2

El algoritmo de Huffman permite optimizar la representación de datos al asignar códigos más cortos a los elementos más frecuentes. En este caso, se logra una reducción significativa del espacio utilizado, alcanzando un ahorro aproximado del 74.35%. Esto demuestra la eficiencia del enfoque greedy en problemas de compresión, especialmente cuando existen diferencias marcadas en las frecuencias de los elementos.

12. PROBLEMA 3 – KNAPSACK 0/1 (Programación Dinámica)

Tienes \$10M para invertir. Proyectos disponibles:

Servidor	Rendimiento	Costo
 HealthTech	\$7M	\$3M
 AI Startup	\$9M	\$5M
 GreenTech	\$4M	\$2M
 Fintech	\$6M	\$4M

- Construye y llena la tabla dp completa
- Identifica la cartera de inversión óptima
- Implementa el backtracking para hallar los proyectos elegidos

12.1 ¿Cómo funciona?

Se construye una tabla donde:

- Filas → proyectos
- Columnas → capacidad (0 a 10)
- Cada celda guarda el **máximo ROI posible**

12.2 Fórmula clave

Cuando el proyecto cabe:

$$dp[i][w] = \max(dp[i-1][w], valor_i + dp[i-1][w - costo_i])$$

Significa:

- NO tomar el proyecto
- O tomarlo y sumar su valor

12.3 Tabla

Proyecto \ Capacidad	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
HealthTech (3,7)	0	0	0	7	7	7	7	7	7	7	7
AI Startup (5,9)	0	0	0	7	7	9	9	9	16	16	16
GreenTech (2,4)	0	0	4	7	7	11	11	13	16	16	20
Fintech (4,6)	0	0	4	7	7	11	11	13	16	16	20

12.4 Explicación de algunas celdas clave

- $dp[2][8] = 16$
HealthTech + AI Startup $\rightarrow 7 + 9$
- $dp[3][10] = 20$
HealthTech + AI Startup + GreenTech $\rightarrow 7 + 9 + 4$

12.5 Resultado óptimo

ROI máximo = 20

12.6 Backtracking (cómo se obtiene la solución)

Desde la última celda:

1. $dp[4][10] = 20 \rightarrow$ igual que $dp[3][10] \rightarrow$ **NO** Fintech no se toma
2. $dp[3][10] \neq dp[2][10] \rightarrow$ **SI** GreenTech
3. $dp[2][8] \neq dp[1][8] \rightarrow$ **SI** AI Startup
4. $dp[1][3] \neq dp[0][3] \rightarrow$ **SI** HealthTech

12.7 Proyectos seleccionados

- ✓ HealthTech
- ✓ AI Startup
- ✓ GreenTech
- Costo total = 10
- ROI total = **20**

12.8 Código

```
Problema 3.py > ...
1 def knapsack(costos, valores, capacidad, nombres):
2     n = len(valores)
3     # 1. CREACIÓN DE LA TABLA DP
4     # Filas: representan los proyectos disponibles (del 0 al n)
5     # Columnas: representan la capacidad de presupuesto (de 0 a 'capacidad')
6     dp = [[0]*(capacidad+1) for _ in range(n+1)]
7
8     # 2. LLENADO DE LA TABLA (Construcción de la solución óptima)
9     for i in range(1, n+1):
10         for w in range(capacidad+1):
11             # Si el costo del proyecto actual cabe en el presupuesto actual 'w'
12             if costos[i-1] <= w:
13                 # Decidimos: ¿Qué da más valor?
14                 # A) No incluir el proyecto: dp[i-1][w]
15                 # B) Incluir el proyecto: valor actual + el mejor valor posible con el resto del dinero
16                 dp[i][w] = max(
17                     dp[i-1][w],
18                     valores[i-1] + dp[i-1][w-costos[i-1]]
19                 )
20             else:
21                 # Si el costo supera 'w', no podemos elegirlo; heredamos el valor anterior
22                 dp[i][w] = dp[i-1][w]
23
24     # 3. BACKTRACKING (Identificar qué proyectos se eligieron)
25     w = capacidad
26     seleccion = []
27
28     # Recorremos la tabla desde el final hacia el principio
29     for i in range(n, 0, -1):
30         # Si el valor cambió respecto a la fila anterior, significa que incluimos este proyecto
31         if dp[i][w] != dp[i-1][w]:
32             seleccion.append(nombres[i-1])
33             # Restamos el costo del proyecto elegido al presupuesto disponible
34             w -= costos[i-1]
35
36     # Retornamos la tabla completa y la lista de proyectos (invertida para que se lea en orden)
37     return dp, seleccion[::-1]
38
39
40 # --- Datos de Inversión ---
41 nombres = ["HealthTech", "AI Startup", "GreenTech", "Fintech"]
42 costos = [3, 5, 2, 4] # Ejemplo: Millones de dólares
43 valores = [7, 9, 4, 6] # Ejemplo: Retorno esperado
44 capacidad = 10 # Presupuesto total disponible
45
46 tabla, seleccion = knapsack(costos, valores, capacidad, nombres)
47
48 print("ROI máximo:", tabla[-1][-1]) # El último valor de la tabla es la solución óptima
49 print("Proyectos elegidos:", seleccion) 🐛
```


12.9 Complejidad

- **Tiempo:** $O(n \times W)$
- **Espacio:** $O(n \times W)$

12.10 Análisis Problema 3

El uso de programación dinámica permite resolver el problema de la mochila 0/1 de manera óptima, evaluando todas las combinaciones posibles sin incurrir en un costo computacional exponencial. En este caso, la solución óptima alcanza un retorno de 20 millones, seleccionando tres proyectos dentro del presupuesto disponible. Este enfoque garantiza la optimalidad de la solución, a diferencia de los algoritmos greedy, que no siempre producen resultados óptimos en este tipo de problemas.

13. Problema 4 – LCS (Subsecuencia Común Más Larga)

Dos versiones de un archivo de configuración:

v1: "DEPLOY-PROD-DB-01"

v2: "DEVELOP-DEBUG-01"

- Halla la LCS entre ambas cadenas
- Muestra la tabla dp completa
- Reconstruye la subsecuencia mediante backtracking
- ¿Qué significa esta LCS en el contexto del versionado?

13.1 Idea del Algoritmo

La **LCS (Longest Common Subsequence)** busca:

- ✓ La secuencia más larga de caracteres que aparece en ambas cadenas en el mismo orden, aunque no necesariamente consecutivos.

Se utiliza **programación dinámica**.

13.2 Regla del algoritmo

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{si } X_i = Y_j \\ \max(dp[i-1][j], dp[i][j-1]) & \text{si } X_i \neq Y_j \end{cases}$$

13.3 Cadenas

v1 = DEPLOY-PROD-DB-01

v2 = DEVELOP-DEBUG-01

13.4 Resultado de la LCS

La subsecuencia común más larga es: DELO-DB-01

Longitud = 10

13.5 Tabla DP

	0	D	E	V	E	L	O	P	-	D	E	B	U	G	-	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
E	0	1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
P	0	1	2	2	2	2	2	3	3	3	3	3	3	3	3	3	3
L	0	1	2	2	2	3	3	3	3	3	3	3	3	3	3	3	3
O	0	1	2	2	2	3	4	4	4	4	4	4	4	4	4	4	4
Y	0	1	2	2	2	3	4	4	4	4	4	4	4	4	4	4	4
-	0	1	2	2	2	3	4	4	5	5	5	5	5	5	5	5	5
P	0	1	2	2	2	3	4	5	5	5	5	5	5	5	5	5	5
R	0	1	2	2	2	3	4	5	5	5	5	5	5	5	5	5	5
O	0	1	2	2	2	3	4	5	5	5	5	5	5	5	5	5	5
D	0	1	2	2	2	3	4	5	5	6	6	6	6	6	6	6	6
-	0	1	2	2	2	3	4	5	6	6	6	6	6	6	7	7	7
D	0	1	2	2	2	3	4	5	6	7	7	7	7	7	7	7	7
B	0	1	2	2	2	3	4	5	6	7	7	8	8	8	8	8	8
-	0	1	2	2	2	3	4	5	6	7	7	8	8	8	9	9	9
0	0	1	2	2	2	3	4	5	6	7	7	8	8	8	9	10	10
1	0	1	2	2	2	3	4	5	6	7	7	8	8	8	9	10	11

13.6 Backtracking (Reconstrucción)

Desde la última celda:

- 1 / ✓
- 0 / ✓
- - / ✓
- B / ✓
- D / ✓
- - / ✓
- O / ✓
- L / ✓
- E / ✓
- D / ✓

Resultado final: DELO-DB-01

13.7 Código

```
Problema 4.py > ...
1  def lcs(X, Y):
2      # m y n son las longitudes de las cadenas v1 y v2
3      m, n = len(X), len(Y)
4
5      # 1. CREACIÓN DE LA MATRIZ DP (Dynamic Programming)
6      # Creamos una tabla de (m+1) x (n+1) llena de ceros.
7      # El +1 es para manejar el caso de una cadena vacía.
8      dp = [[0]*(n+1) for _ in range(m+1)]
9
10     # 2. LLENADO DE LA TABLA
11     for i in range(1, m+1):
12         for j in range(1, n+1):
13             # Si los caracteres actuales coinciden:
14             if X[i-1] == Y[j-1]:
15                 # Tomamos el valor de la diagonal (arriba-izquierda) y sumamos 1
16                 dp[i][j] = dp[i-1][j-1] + 1
17             else:
18                 # Si no coinciden, heredamos el valor máximo entre:
19                 # El de arriba (dp[i-1][j]) o el de la izquierda (dp[i][j-1])
20                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
21
22     # 3. BACKTRACKING (Reconstrucción del resultado)
23     # Al terminar el bucle, dp[m][n] tiene la longitud de la LCS.
24     # Ahora recorremos la tabla hacia atrás para saber qué letras elegimos.
25     i, j = m, n
26     subsecuencia = ""
27
28     while i > 0 and j > 0:
29         # Si los caracteres son iguales, esta letra es parte de la LCS
30         if X[i-1] == Y[j-1]:
31             subsecuencia = X[i-1] + subsecuencia
32             i -= 1
33             j -= 1
34         # Si no, nos movemos en la dirección del valor más grande en la tabla
35         elif dp[i-1][j] > dp[i][j-1]:
36             i -= 1
37         else:
38             j -= 1
39
40     return dp, subsecuencia
41
42     # --- Ejemplo de uso ---
43     v1 = "DEPLOY-PROD-DB-01"
44     v2 = "DEVELOP-DEBUG-01"
45
46     tabla, lcs_resultado = lcs(v1, v2)
47
48     print("LCS:", lcs_resultado)
49     # Resultado esperado: "DE-D-01" (los caracteres comunes en orden)
```

13.8 ¿Qué significa esta LCS en el contexto del versionado?

La LCS representa:

- Los fragmentos que permanecen comunes entre ambas versiones del archivo.

En este caso:

- DEL O - DB - 01

Indica que ambas configuraciones comparten:

- prefijos similares (**DE...**)
- estructura con separadores -
- referencia a base de datos **DB**
- versión o identificador final **01**

Esto ayuda a detectar:

- cambios entre versiones
- partes reutilizadas
- diferencias exactas
- evolución del archivo

Es la base conceptual de herramientas como **Git diff**, comparación de archivos y merge automático.

13.9 Complejidad

- **Tiempo:** $O(m \times n)$
- **Espacio:** $O(m \times n)$

13.10 Análisis del Problema 4

El algoritmo LCS permitió identificar la subsecuencia común más larga entre "**DEPLOY-PROD-DB-01**" y "**DEVELOP-DEBUG-01**", obteniendo como resultado "**DELO-DB-01**". Esto indica que ambas versiones conservan varios elementos en común, aunque presentan cambios en algunas partes. En el contexto del versionado, la LCS representa la información que se mantiene entre versiones, mientras que lo demás corresponde a modificaciones o nuevas configuraciones. Este método es útil para comparar archivos y detectar cambios de forma automática. La programación dinámica permite resolver el problema de manera eficiente con una complejidad de $O(m \times n)$.

14. Referencias

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Python*. Wiley.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson Education.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Python Software Foundation. (2026). *Python language reference, version 3.x*.
<https://www.python.org/>

Git. (2026). *Git documentation*. <https://git-scm.com/doc>

OpenAI. (2026). *ChatGPT*. <https://chat.openai.com/>